Distributed Multi-GPU Sparse Matrix-Vector Multiplication (SpMV) via 1D Cyclic Partitioning and GPU-Aware Communication

Michael Bernasconi
Student ID: 267681
michael.bernasconi@studenti.unitn.it
https://github.com/Michael-Bernasconi/Sparse-Matrix-Vector-Multiplication/tree/Deliverable2
Department of Information Engineering and Computer Science
University of Trento

***Abstract*—Distributed Sparse Matrix-Vector multiplication (SpMV) represents a vital bottleneck in massive-scale numerical computing, where efficiency is heavily constrained by structural irregularity, network latency, and load imbalance. This study extends single-node architectures to a multi-GPU environment by evolving a continuous block partition layout into a 1D cyclic distribution ($owner(i) = i \pmod P$). By completely removing high-overhead global collective broadcasts for vector data distribution, a custom point-to-point sparse communication overlay is implemented. This layout tracks, isolates, and exchanges exclusively the non-local active components of the multiplier vector x (Ghost Entries) through non-blocking peer-to-peer routines. Furthermore, intermediate host-staged buffer paths are eliminated by refactoring the pipeline with GPU-Aware MPI paradigms operating directly on device memories. Performance evaluations conducted on the University of Trento cluster (Ampere A30 GPUs) demonstrate that while 1D cyclic mapping perfectly balances uniform data distributions (e.g., ASIC structures), highly skewed power-law structures remain strictly bound by communication overhead, providing clear experimental insights regarding the intersection of data topology, interconnect bandwidth, and multi-GPU scalability.**

***Index Terms*—SpMV, Multi-GPU CUDA, 1D Cyclic Partitioning, Ghost Entries Exchange, GPU-Aware MPI, Load Balancing, Performance Interconnect Bound, Strong Scaling, Weak Scaling.**

I. Introduction

Sparse Matrix-Vector multiplication ($y = A \times x$) is the underlying backbone of modern scientific applications, differential equation solvers, and graph analytics. Evolving single-node GPU kernels to distributed architectures introduces severe scalability roadblocks caused by non-contiguous memory layouts and network bottlenecks. The original project codebase relied on a standard 1D contiguous row-block partitioning scheme. This approach, however, proved highly inefficient when dealing with real-world matrices from the SuiteSparse collection, as their non-uniform sparsity patterns led to significant computational idle time (load imbalance) and heavy memory overhead due to redundant vector replication.

To overcome these limitations, this work introduces a 1D cyclic row-distribution framework designed to optimally balance parallel workloads across multiple hardware devices. By rewriting the communication infrastructure around localized peer-to-peer sparse exchanges, the system completely eliminates collective broadcast overheads. This report details the core design principles, structural transformations, and benchmarking outcomes of the distributed SpMV execution campaign across both strong and weak scaling configurations.

II. Parallel Design Methodology

A. Evolution via Foster's Design Strategy

To effectively transition single-device execution paths into a multi-GPU paradigm, the application architecture was evolved adhering strictly to Foster's four-stage design methodology:

- Partitioning: Fine-grained domain decomposition is performed on the rows of matrix A. Rank 0 sequentially reads the Matrix Market file and distributes entries to the target cluster ranks following a modular distribution scheme: *owner(i) = i (mod size)*
  where i represents the global row index and size corresponds to the number of active MPI ranks (GPUs).
- Communication: Rather than distributing the entire multiplier vector x via global collectives, a specialized discovery process identifies non-local indices requested by local non-zero elements (col_idx). These indices represent the matrix *Ghost Entries*, which are captured in isolated communication descriptors.
- Agglomeration: Rows assigned to a single executor are agglomerated into an independent, compact local sparse matrix representation (CSR or COO layouts), converting the sparse structure into relative coordinate systems to ensure zero index-translation overhead during raw kernel execution.
- Mapping: Each MPI process is pinned to a dedicated hardware device ($NVIDIA\ A30\ GPU$), executing localized data processing routines natively within isolated CUDA streams.

B. GPU-Aware Point-to-Point Communication Overlay

To maximize throughput, intermediate host-side staging is bypassed. While standard multi-GPU setups copy device results to host buffers via high-latency `cudaMemcpy` prior to MPI calls, this architecture leverages a GPU-Aware MPI runtime. Device pointers are passed directly to non-blocking primitives (`MPI_Irecv` and `MPI_Isend`), followed by an `MPI_Waitall` barrier.The complete execution sequence is formally outlined in Algorithm 1. This approach targets the minimization of data transmission overhead, which has been widely recognized as the primary performance bottleneck in multi-GPU SpMV implementations [2].

Algorithm 1: Distributed GPU-Aware SpMV Pipeline

Input: A_{loc} (local matrix partition), x_{loc} (local vector partition), I (benchmark iterations)
Output: y_{global} (full result vector assembled on Root)
Identify ghost cells and allocate `d_ghost_*`
`pack_ghost_kernel<<<...>>>;` `cudaDeviceSynchronize()`
`MPI_Irecv(...)`; `MPI_Isend(...)`; `MPI_Waitall(...)`
`unpack_ghost_kernel<<<...>>>`; Start `cudaEvent` timers
for $iter \leftarrow 1$ **to** I **do**
  `cudaMemset(d_y, 0, ...)`; Launch SpMV Kernel; `cudaDeviceSynchronize()`
end
Stop timers; `MPI_Gatherv(d_y, ..., Root)`
if Rank $== 0$ **then**
  Reassemble `d_interleaved_y` $\rightarrow$ linear y_{global}
end

III. Experimental Setup & Dataset

A. Hardware Infrastructure

All execution cycles were evaluated on the `edu01` compute node (`edu-short` partition) of the University of Trento cluster. The exact hardware specifications are detailed in (Table I). All configurations utilized CUDA v12.3.2 and OpenMPI v4.1.5 with native CUDA-Aware support enabled.

TABLE I: Target Hardware Specifications

Component	Host (CPU)	Device (GPU)
Model	Intel(R) Xeon(R) Silver 4309Y	NVIDIA A30
Architecture	Ice Lake (x86_64)	Ampere (Compute Cap. 8.0)
Cores / SMs	16 Cores / 32 Threads (2 Sockets)	56 SMs (3584 CUDA Cores)
Frequency	2.80 GHz (Base) / 3.60 GHz	1.44 GHz (Boost)
FP32 Perf.	1.84 TFLOPS	10.3 TFLOPS
Memory BW	102.4 GB/s	933.1 GB/s (HBM2)
Global Mem.	N/A	24 GB
Shared Mem.	N/A	48-100 KiB /SMs
Warp Size.	N/A	32

B. Dataset Optimization Strategy

To guarantee a robust performance profile covering both Strong and Weak Scaling benchmarks, the experimental matrix framework is divided into two categories: real-world structures for Strong Scaling (TableII), and algorithmically generated structures for Weak Scaling (TableIII).

At the same time, the dataset has been revised to avoid memory allocation failures on the cluster while maintaining high topological variance and uneven sparsity domains.

1) Strong Scaling Dataset: The baseline dataset comprises ten distinct real-world sparse structures from the SuiteSparse Matrix Collection, carefully selected to evaluate the algorithms against different structural domains.

TABLE II: Real Datasets (Strong Scaling Configurations)

ID	Matrix Name	Rows (M)	Columns (N)	NNZ	Avg. NNZ	Dev. Std.	Structural Domain / Description
1420	ASIC_680ks	682,712	682,712	1,693,767	2.48	4.16	Circuit simulation. Highly sparse and asymmetric distribution.
1297	boyd2	466,316	466,316	1,500,397	3.22	194.91	Convex optimization. Extreme structural imbalance (dense spikes).
1881	Rucci1	1,977,885	109,900	7,791,168	3.94	0.34	Least squares. Tall rectangular layout, perfectly uniform rows.
2379	webbase-1M	1,000,005	1,000,005	3,105,536	3.11	25.35	Web crawl graph. Power-law scaling with massive hub nodes.
1419	ASIC_320ks	321,671	321,671	1,316,085	4.09	3.80	Circuit simulation. Medium-sized uniform sparse test.
1513	patents_main	240,547	240,547	560,943	2.33	1.94	Patent citation network. Ultra-lightweight and predictable 1D blocks.
39	bcsstk17	10,974	10,974	428,650	39.06	8.40	Structural stiffness matrix. Small size but high row density.
1269	m_t1	185,000	97,500	9,753,570	52.72	1.18	Least squares. Rectangular grid, exceptionally well-balanced.
38	bcsstk16	4,884	4,884	290,378	59.45	19.23	Structural problem. Smallest test, high density with localized noise.
1416	ASIC_100ks	99,190	99,190	578,890	5.84	13.91	Circuit simulation. Small regular baseline for hardware verification.

2) Weak Scaling Dataset (Synthetic Generation): To properly evaluate Weak Scaling, the problem size must grow proportionally to the number of computational resources (N, $2N$, $4N$). It has been developed a custom Python generator creating two distinct synthetic topologies with a constant $D_{avg} \approx 3.0$:

- ASIC-like (`synth_asic`): Highly sparse but structurally uniform. Ensures perfect 1D cyclic load balancing while maintaining randomized non-zero column placements to simulate cache-miss irregularities.
- BOYD2-like (`synth_boyd2`): Designed for extreme load imbalance. Exactly 2% of the rows absorb 75% of the total Non-Zeros, mimicking power-law hub topologies and severely punishing 1D distribution schemes.

TABLE III: Synthetic Datasets (Weak Scaling Configurations)

Target GPUs	Matrix Name	Rows (M)	Columns (N)	Total NNZ	Avg. NNZ	Structural Profile
1 GPU	synth_asic_1gpu	200,000	200,000	$\approx$ 600,000	3.0	Balanced / Uniform
1 GPU	synth_boyd2_1gpu	200,000	200,000	600,000	3.0	Imbalanced / Hubs
2 GPUs	synth_asic_2gpu	400,000	400,000	$\approx$ 1,200,000	3.0	Balanced / Uniform
2 GPUs	synth_boyd2_2gpu	400,000	400,000	1,200,000	3.0	Imbalanced / Hubs
4 GPUs	synth_asic_4gpu	800,000	800,000	$\approx$ 2,400,000	3.0	Balanced / Uniform
4 GPUs	synth_boyd2_4gpu	800,000	800,000	2,400,000	3.0	Imbalanced / Hubs

IV. Performance Evaluation

Execution statistics were gathered across 1, 2, and 4 GPU configurations. Custom execution frameworks (Distributed CSR, COO, CSR-Vector and cuSPARSE) were benchmarked against the `SpMV Baseline`.

A. Execution Time and Computational Throughput

As shown in Figs. 1 and 2, `cuSparse` dominates in raw speed. However, the `CSR-Vector` and `CSR` kernel excels, peaking near 300 GFLOPS.

B. Strong Scaling: Speedup and Efficiency

As visualized across the real datasets in Figures 3 and 4, the Strong Scaling capabilities display clear behavioral trends among the different configurations. While adding additional GPUs introduces non-trivial network overhead, the `CSR-Vector` approach scales the best among custom solutions, yielding an average speedup of $2.69\times$ on 4 GPUs with an efficiency of roughly 67.4%. Conversely, the `SpMV Baseline` implementation is inherently a sequential host-bound emulation executing strictly on a single CPU core (Rank 0). Since it cannot execute on distributed hardware, its metrics are explicitly zeroed out for the 2 and 4 GPU configurations in Figs. 3 and 4 to accurately reflect that the multi-GPU cluster infrastructure remains completely unutilized by the sequential context.

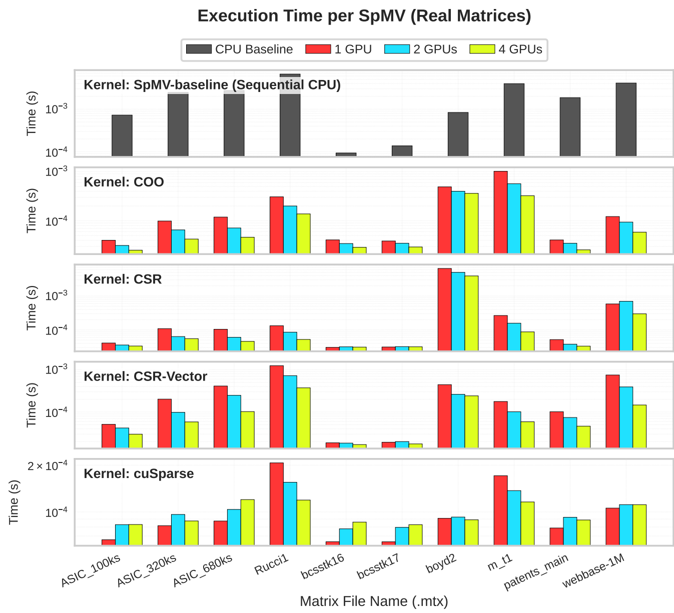

Fig. 1: Execution Time (s) per SpMV on Real Matrices (Log Scale).

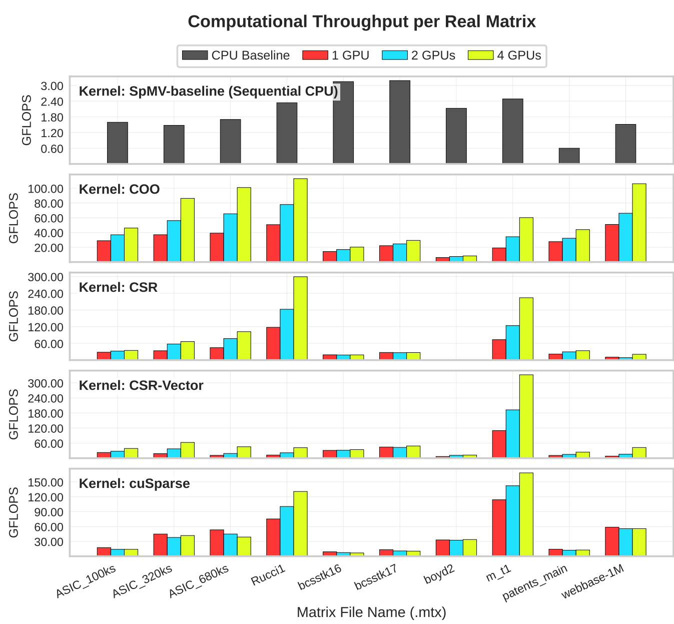

Fig. 2: Computational Throughput (GFLOPS) per Real Matrix.

C. Communication vs Computation Breakdown

A crucial aspect of distributed GPU programming is understanding network synchronization costs. Figure 5 isolates MPI interactions for the true multi-GPU parallel kernels, completely excluding the sequential `SpMV Baseline` since it features no network overhead. The profiling highlights that the massive relative cost of distributed coordination dominates the multi-device execution. Since local SpMV kernels execute in microseconds, packing buffers, marshaling data over PCIe/NVLink, and waiting for peer-to-peer completion far exceed pure computation. On 4 GPUs, the implementations are completely communication-bound: the Ghost Exchange

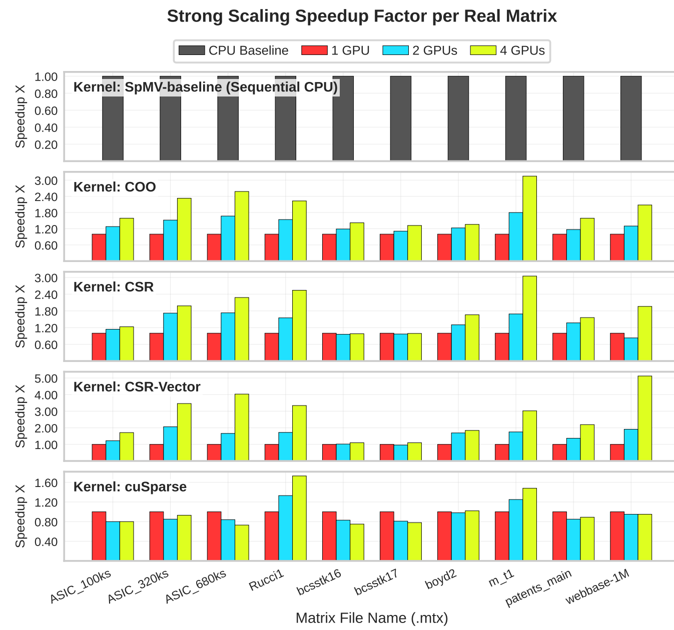

Fig. 3: Strong Scaling Speedup Factor per Real Matrix.

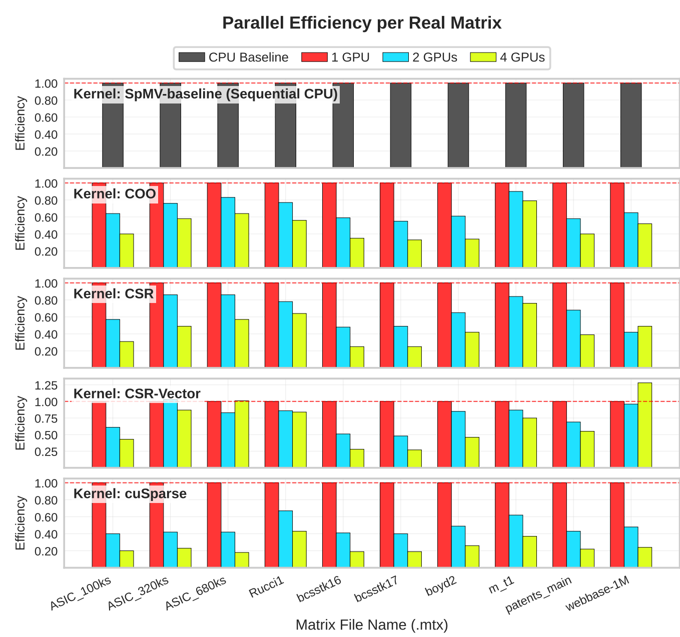

Fig. 4: Parallel Efficiency per Real Matrix (1, 2, and 4 GPUs).

overhead is $43.9\times$ higher than pure compute for `COO`, and $32.7\times$–$33.5\times$ for `CSR` variants. Even `cuSparse` is heavily penalized ($20.9\times$). Increasing the GPU count yields diminishing returns as runtime is limited by interconnect latencies.

D. Weak Scaling Profiling

To isolate the structural impact on multi-node balancing, Weak Scaling was evaluated using synthetic datasets (Fig. 6). The custom implementations maintain stable and robust scaling efficiency on the `synth_asic` family (where 1D cyclic distribution helps distribute rows), reaching 0.81 efficiency on 4 GPUs. The `synth_boyd2` datasets maintain a comparable

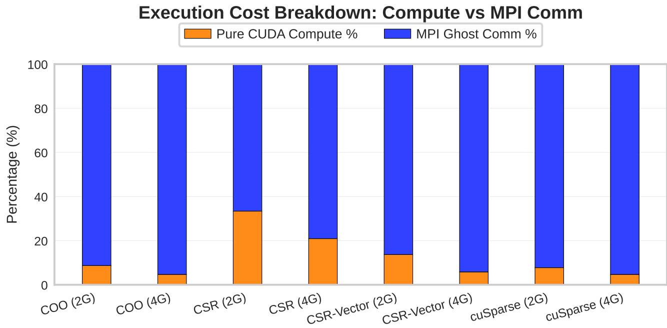

Fig. 5: Execution Cost Breakdown: Pure CUDA Compute vs. MPI Ghost Communication Overhead.

efficiency of 0.79 on 4 GPUs, proving that the 1D cyclic scheme successfully limits computational load imbalance under weak scaling.

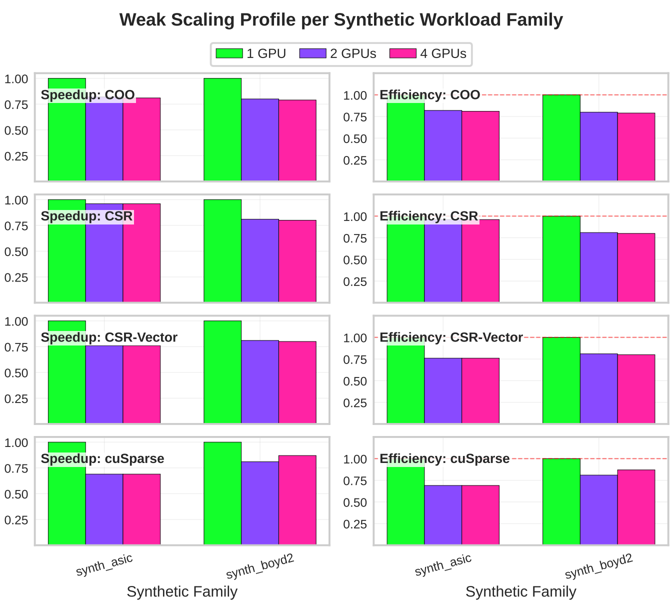

Fig. 6: Weak Scaling Efficiency and Speedup on Synthetic Workloads.

V. Discussion

A. When 1D Partitioning Works Well

1D cyclic partitioning delivers solid execution scaling when applied to synthetic matrices under weak scaling workloads, as shown by the `synth_asic` and `synth_boyd2` families, which retain around 81% and 79% parallel efficiency respectively. Because rows are distributed in an interleaved manner, variations in non-zero density can be partially filtered out, helping ensure that each independent GPU node receives a balanced load under weak scaling contexts.

B. When 2D Partitioning is Advantageous

Although the 1D cyclic scheme demonstrates surprising resilience on synthetic workloads under weak scaling, it fundamentally struggles when strong scaling highly skewed power-law graphs or extreme optimization structures like `boyd2` ($\sigma = 194.91$). These datasets contain hyper-dense rows that act as severe bottlenecks. Because a 1D allocation scheme assigns an entire row indivisibly to a single executor, the GPU receiving the hyper-dense rows becomes a critical performance limiter. In this scenario, a 2D partitioning strategy is highly advantageous to break up highly connected hubs across multiple executors. As structurally argued in literature, advanced two-dimensional graph layouts and edge partitioning provide significant scaling improvements over 1D strategies when mapping scale-free networks onto thousands of cores [1].

C. When the Implementation is Bound by the Interconnect

As shown in the cost breakdown (Fig. 5), the implementation becomes strictly bound by the interconnect network when scaling to 4 GPUs. The time required by `MPI_Isend` and `MPI_Irecv` routines to exchange non-local elements over the PCIe/network interface easily exceeds the pure execution time of the CUDA sparse multiplication kernel by factors up to $44\times$. Under these conditions, scaling to 4 GPUs yields decreasing performance returns, as the communication overhead dominates the total Time To Solution. This performance limit is confirmed by recent literature, showing that data transfer dominates SpMV runtime on 4-GPU configurations [2].

D. Largest Manageable Matrix Size

With the current memory layout, the maximum manageable matrix size is bounded primarily by the available memory on a single GPU node (NVIDIA A30 24GB). By replacing the global vector broadcast mechanism with a localized Ghost Entry exchange, this implementation avoids duplicating the full multi-million-element multiplier vector x on every device, allowing the cluster to easily handle large datasets exceeding 50 million non-zero elements per node context.

VI. Conclusion

This study expanded custom sparse matrix execution paths into a highly optimized distributed multi-GPU paradigm. Implementing a 1D cyclic distribution pattern effectively helps balance loads under weak scaling configurations. Transitioning to a sparse peer-to-peer point-to-point network model built on GPU-Aware MPI concepts significantly reduces communication overheads compared to standard collective broadcasts, though it remains heavily communication-bound at higher GPU counts. Ultimately, experimental results underscore the critical importance of matching data topology with appropriate parallel distribution models, as highly skewed matrices remain limited by interconnect performance and strong scaling constraints under 1D limitations.

References

[1] E. G. Boman, K. D. Devine, and S. Rajamanickam, "Scalable Matrix Computations on Large Scale-Free Graphs Using 2D Graph Partitioning," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC13)*, 2013.

[2] J. Gao, W. Ji, and Y. Wang, "Optimization of Large-Scale Sparse Matrix-Vector Multiplication on Multi-GPU Systems," *ACM Transactions on Architecture and Code Optimization (TACO)*, vol. 21, no. 4, art. 69, 2024.